

gitkit CLI

純 bash 小工具,每個指令其實只是幾個日常 git 指令的組合。

這份投影片把 `st · ci · up · diff · reset · exp` 背後實際跑的原生 git 流程一頁一頁攤開。

目標環境:CentOS 7 · git **1.8.3.1** · bash 4.2 · 不需 root

翻頁   /  · 列印成 PDF:瀏覽器  + 

設計原則 — 只用「看得懂的」原生 git

沒有自創魔法。每個 gitkit 指令 = 收集狀態 → 選單挑選 → 跑標準 git。全部指令在 git 1.8.3.1 都存在,不用任何冷僻 plumbing。

`gitkit st` → `git status`

`gitkit reset` → `git reset`

`gitkit ci` → `add` · `commit` · `fetch` · `merge` · `push`

`gitkit exp` → `git archive` / `git show`

`gitkit up` → `stash` · `fetch` · `merge` · `stash pop`

`gitkit log` → `git log --name-status`

`gitkit diff` → `git difftool`

輔助讀取(唯讀,挑選用): `git status --porcelain` · `git branch / branch -r` · `git log --oneline` · `git symbolic-ref --short HEAD` · `git rev-parse @{upstream}`

gitkit st — 看狀態

最薄的一層,直接轉呼叫 `git status` 。

1

gitkit st — 完整狀態

```
git status
```

2

gitkit st -uq — 只看已修改 / 已暫存(隱藏未追蹤)

```
git status -uno  
# u-no = untracked files: no
```

其他參數會原樣轉給 `git status` ,例如 `gitkit st -s` → `git status -s` 。

gitkit ci — ① 挑檔 → 提交 → 選分支

- 1 收集工作區狀態,分成 U / M / S 三類

```
git status --porcelain
```

- 2 選單挑出要提交的檔 → **add**

```
git add -- <選到的檔案>
```

- 3 確認暫存區有東西 → 輸入訊息 **commit**

```
git diff --cached --quiet # 有暫存才提交  
git commit -m "<訊息>"
```

- 4 列出本地 + 遠端分支,挑「要整合並推送到」哪一支

```
git branch # 本地  
git branch -r # 遠端
```

沒有任何變更時 → 顯示 No changes to commit ,並詢問是否仍要同步+推送。

gitkit ci — ② 暫存 → 整合 → 推送

- 5 若工作區還有沒選到的修改 → 先 **stash** 收起來(保持合併乾淨)

```
git diff --quiet    # 髒的話 ↓  
git stash
```

- 6 取回遠端,把選定分支**併進**當前分支

```
git fetch <remote>      # 抓全部 ref(1.8.3.1 才會更新 origin/*)  
git merge --no-edit <branch>
```

- 7 把剛剛 stash 的修改**還原**回來

```
git stash pop
```

- 8 確認後,把當前 HEAD **推送**到選定分支

```
git push <remote> HEAD:<branch>
```

第 6 步的 merge、第 7 步的 stash pop,任一發生衝突 → 進入下一頁的**衝突處理迴圈**。

衝突處理 — 七個選項各對應什麼 git

先用 `git diff --name-only --diff-filter=U` 找出所有衝突檔,再對全部套用你選的處理:

選項	意義	底層 git
<code>tf</code>	their full — 整檔用對方版	<code>git checkout --theirs -- f && git add -- f</code>
<code>mf</code>	mine full — 整檔用我方版	<code>git checkout --ours -- f && git add -- f</code>
<code>tc</code> / <code>mc</code>	只在衝突區塊取一側 (保留自動合併的部分,支援 diff3)	<code># awk 解析 <<< === >>> 標記</code> <code>awk ... f > tmp && mv tmp f && git add -- f</code>
<code>e</code>	edit — 開你設定的 mergetool	<code>git mergetool</code>
<code>r</code>	resolved — 標記為已解決	<code>git add -- f</code> # 仍含標記者會擋下
<code>a</code>	abort — 放棄	<code>git merge --abort</code> # stash 情境則 <code>git reset --hard HEAD</code>

全部解完 → merge 情境補一筆 `git commit --no-edit ;` stash 情境則 `git stash drop` 。

gitkit up — 只同步,不提交/不推送

把當前分支更新到它的 upstream。就是 ci 的「stash → 整合 → 還原」那一段。

1

找出當前分支的 upstream

```
git symbolic-ref --short -q HEAD      # 當前分支  
git rev-parse --abbrev-ref @{upstream} # 對應遠端(沒有就報錯)
```

2

有修改 → 先 stash

```
git stash # 若 git diff --quiet 為真
```

3

取回 + 併入 upstream(衝突 → 衝突處理迴圈)

```
git fetch <remote>  
git merge --no-edit <remote>/<branch>
```

4

還原 stash(衝突 → 衝突處理迴圈)

```
git stash pop
```

gitkit diff — 用你設定的工具看差異

列出 U/M/S 讓你挑,選到的依序丟給 `git difftool` (走 git 標準 `diff.tool` 設定)。

1

收集並分類變更檔

```
git status --porcelain
```

2

已暫存的檔(S)→ 比對「暫存區 vs HEAD」

```
git difftool --cached -- <file>
```

3

已修改的檔(M)→ 比對「工作區 vs 暫存區」

```
git difftool -- <file>
```

未追蹤檔(U)沒有前一版本,會被略過。先設定: `git config diff.tool <tool>`。

gitkit reset — 取消暫存,或回到某個 commit

A

取消暫存 — 從已暫存清單挑檔,退回工作區(內容不動)

```
git status --porcelain      # 取得已暫存清單
git reset -q HEAD -- <files>
```

B

重置到某個 commit — 從近 20 筆挑一筆,再選模式

```
git log --oneline -n 20      # 挑 SHA
git reset --soft|--mixed|--hard <sha>
```

--soft 保留索引+工作目錄 · **--mixed**(預設)保留工作目錄、清索引 · **--hard** 丟棄所有未提交變更(危險,會先要你確認)。

gitkit exp — 取出不帶 .git 的資料夾

把板控中的某個資料夾/檔案,內容平鋪匯出到 `dest` (預設 = `<path>_exp`)。

1

判斷這個路徑是資料夾(tree)還是單檔(blob)

```
git cat-file -t HEAD:<path> # → tree / blob / 不存在則報錯
```

2

資料夾 → 以該子樹為根 archive 成 zip,解壓進 dest(內容平鋪)

```
git archive --format=zip -o tmp.zip HEAD:<path>
unzip -o -q tmp.zip -d <dest> && rm tmp.zip
```

2'

單一檔案 → 直接寫出(免 unzip)

```
git show HEAD:<path> > <dest>/<name>
```

用 `HEAD:<path>` 當 archive 的根,所以解出來直接是內容,不會多一層 `<path>/`。

gitkit log — SVN 使用者看這裡

`svn log -v` 的 git 版:每筆 commit 附上動到的檔案(A/M/D)。

svn	意義	git
<code>log</code>	看歷史	<code>git log</code>
<code>-v</code>	列出變更的路徑	<code>--name-status</code>
<code>-r A:B</code>	限定版本範圍	<code>A..B</code> (commit 範圍)
<code>-l N</code>	只取 N 筆	<code>-n N</code>
<code><file></code>	只看這個檔	<code>-- <file></code>

```
# svn log -v -r A:B -l N <file> 等於
git log --name-status A..B -n N -- <file>
# 日常：某檔的詳細歷史
gitkit log <file> [limit] # → git log --name-status -- <file>(預設 20 筆)
```

SVN 的 `rNNN` 是全域整數;git 沒有,改用 commit SHA 或範圍 `A..B` / `HEAD~5..HEAD` / `--since=2026-01-01` 。完整對照見 `cli/docs/svn-to-git.md` 。

整體對照 — 全部用到的原生 git

gitkit	原生 git 流程
st	status / status -uno
ci	status --porcelain → add → commit → branch / branch -r → stash → fetch → merge → stash pop → push HEAD:
up	rev-parse @{upstream} → stash → fetch → merge → stash pop
diff	status --porcelain → difftool [--cached] -- <f>
reset	reset HEAD -- <f> · log --oneline → reset --soft/--mixed/--hard
exp	cat-file -t → archive --format=zip HEAD:<p> + unzip · 單檔 show
log	log --name-status -n <N> -- <file> (= svn log -v)
衝突	diff --diff-filter=U · checkout --ours/--theirs · mergetool · merge --abort · commit --no-edit

相容性: 全程不使用 `git -C` 、不使用 porcelain v2 、不使用 `git stash push (2.13+)`; `fetch <remote>` 取代單分支 `fetch` — 都是為了在 **git 1.8.3.1** 正確運作。